

SYSTEM CALLS

The controlled gateway from user space to the kernel

1 WHY



PROTECTION

Kernel & other processes protected from user code.



ISOLATION

Per-process address space, resources, capabilities.



STABLE HARDWARE ABSTRACTION

Portable API over diverse hardware.



PRIVILEGED OPERATIONS

I/O, device access, interrupt control, memory mgmt.

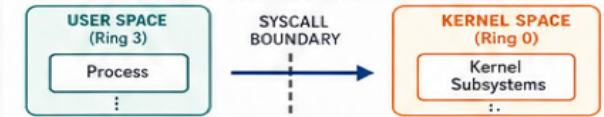
2 CORE IDEA

- 1 User code calls libc/runtime wrapper.
- 2 Wrapper loads syscall number and arguments in registers.
- 3 Execute trap/syscall instruction.
- 4 CPU switches to kernel mode and enters kernel.
- 5 Kernel validates, performs work.
- 6 Kernel places result (or -errno) in return register.
- 7 Return to user mode; wrapper returns result/errno.

user code → wrapper → trap → kernel → work → return → user code

3 INTERNALS

USER SPACE → SYSCALL BOUNDARY → KERNEL

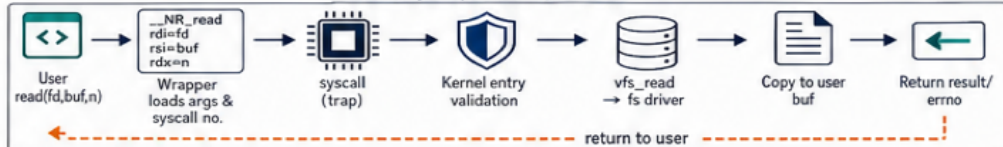


Privilege Rings	Ring 3 (user) → Ring 0 (kernel) on trap.
Syscall Table / Dispatcher	Syscall number indexes table → handler function.
Register ABI	Arch-specific (e.g., x86-64, ARM64). Args in registers; ret in RAX/X0.
Context Save	CPU saves minimal state; may save full context on block/preempt.
Validation & Copy	Check args, permissions; copy_from_user / copy_to_user; handle faults.
Blocking / Wakeup	May sleep (I/O, lock). Scheduler picks another task; wakes on event.
Return Path	Set ret/errno → syscall return → restore user state → resume.

NOTES: Distinguish from...

- Ordinary function call: same mode, no privilege change.
- Interrupt: async from device; not requested by process.
- Exception: fault/trap due to error (e.g., page fault).

READ(FD, BUF, N) PATH (EXAMPLE)



4 COMPLEXITY



Fixed entry/exit overhead (dozens-hundreds of CPU cycles; arch dependent).



Argument validation & access checks.



Cache/TLB effects (kernel code/data, page table walks).



Scheduler & blocking (syscall may sleep; context switch if needed).



I/O dominates cost (disk, network, device). Orders of magnitude slower than CPU.



Batching & amortization reduce syscall frequency.



Zero-copy & in-kernel paths reduce copies & overhead.



A syscall is NOT necessarily a full process context switch. Only when blocking or preempted.

5 TRADE-OFFS

BENEFITS	COSTS
Security & isolation	Boundary crossing latency
Modularity & clear abstractions	Memory copies (user ↔ kernel)
Portability across hardware/OS	Nondeterminism (scheduling, I/O)
Stable, evolvable interfaces	Larger attack surface (input validation)
Resource control & accounting	More code paths & complexity

Design goal: Minimize crossings & copies while preserving safety and correctness.

6 DISCUSSION

- Why privilege separation? Prevent malicious/buggy user code from harming system or others.
- Why libc wrappers? Convenience API, portability, arg marshalling, restart logic (EINTR), vDSO use.
- Syscall vs context switch? Syscall ≠ switch. Switch happens only when blocking/preempted.
- Blocking vs nonblocking? Choose based on latency vs throughput; readiness via select/poll/epoll.
- Async I/O? Threads, AIO, io_uring, completion ports.
- Batching? ready/writer, sendmmsg/recvmmsg, openat2 + RESOLVE_* flags, io_uring SQEs.
- vDSO? User-space calls for gettimeofday, clock_gettime, getcpu, etc. (no kernel trap).
- mmap? Map files/devices to avoid read/write syscalls.
- io_uring? Ring buffers, SQ/CQ, fewer traps, fixed files.
- Zero-copy? sendfile, splice, copy_file_range, MSG_ZEROCOPY.
- Failure modes? EFAULT, EPERM, ENOENT, EINTR, EMFILE, ENOMEM, EAGAIN, partial I/O, timeouts.

7 RELATED



Interrupts: Device → kernel (async).



Exceptions: Faults/traps (e.g., page fault).



Scheduling: Pick next runnable task.



Context Switching: Save/restore task state.



Virtual Memory: Page tables, faults, TLB.



File Descriptors: Per-process handles to objects.



IPC: Pipes, sockets, shared memory, message queues.



Signals: Async notifications to processes.

8 PRODUCTION

- Files: open, read, write, close, fsync, stat.
- Sockets: socket, connect, send, recv, accept.
- Processes: fork, execve, wait, exit, kill.
- Memory Mapping: mmap, munmap, mprotect.
- Clocks/Timers: clock_gettime, nanosleep, timerfd.
- Containers/Sandbox: namespaces, chroot, seccomp, capabilities.
- Observability: strace, perf trace, eBPF, LTTng, SystemTap, auditd.

9 INTERVIEW THINKING

- 1 Clarify operation. What resource? What result expected?
- 2 Trace the boundary. What syscall(s)? Any wrappers?
- 3 Identify blocking & copies. Where can it sleep? Copies user ↔ kernel?
- 4 Estimate costs. Frequency × per-call cost; I/O vs CPU.
- 5 Consider failure & security. Errors? Permissions? Races?
- 6 Improve. Measure → profile → reduce crossings/copies → choose right pattern (batching, async, mmap, io_uring).

10 SENIOR NOTES

- ✓ Optimize measured hot paths; don't guess.
- ✓ Batch work and increase granularity.
- ✓ Handle EINTR, partial I/O, and short reads/writes.
- ✓ Validate trust boundaries; minimize attack surface.
- ✓ Prefer nonblocking + readiness for latency sensitivity.
- ✓ Use timeouts and cancellation.
- ✓ Reason about tail latency, queueing, and backpressure.
- ✓ Prefer portable abstractions unless evidence demands specialization.

11 MENTAL HOOK

“A system call is a guarded door: crossing it is cheap enough to use, but costly enough to design around.”

CPU SCHEDULING

Choosing what runs next—and why

1 WHY



Multiplex scarce CPUs



Responsiveness
(low latency)



Throughput
(high work done)



Fairness
(equitable access)



Deadlines
(real-time guarantees)



Isolation
(noisy neighbor control)

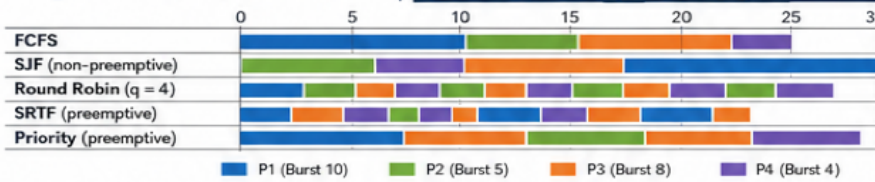
2 CORE IDEA

- Scheduler selects a **RUNNABLE** task from ready queues.
- May **preempt** **RUNNING** task (timer / higher priority / events).
- Task states:



- Scheduling policy encodes goals (throughput, fairness, latency, deadlines...).

GANTT EXAMPLE (same workload)



3 INTERNALS



Timer interrupt: drives preemption & time slice.



Wakeup events: I/O completion, signals, etc.



Per-CPU run queues: typically lock-free; local queues for cache locality.



Priority: static/dynamic; nice, weights, deadlines.



Load balancing: steal/half-split tasks across CPUs to balance load.



Affinity & locality: CPU affinity, cache warmth, core specialization.



Context switch: save/restore registers, PC, stack, TLS, MMU state; flush TLB may occur.



Accounting: runtime, waits, vruntime/nice, statistics for policy & tuning.

Threads:

- User threads (M:N / many-to-many) – runtime scheduler (e.g., Go, JVM).
- Kernel threads (1:1) – OS scheduler.



Multicore:
Per-CPU queues, cross-CPU wakeups, work stealing.



NUMA:
Favor local memory; avoid remote hops; NUMA-aware placement.

4 COMPLEXITY

	Selection cost	$O(1)$ (simple queues) $O(\log n)$ (trees/heaps) $O(n)$ (linear scan)
	Enqueue/dequeue	$O(1)$ typical
	Quantum overhead	$\approx 2 \times$ context switch + timer cost
	Waiting time	Lower is better
	Turnaround time	Completion – arrival
	Response time	First run – arrival
	Throughput	Jobs completed / unit time
	CPU utilization	Busy time / wall-clock
	Deadline misses	Count / %
	Tail latency (p99)	High tails => poor UX

Pitfalls: convoy effect, starvation, priority inversion, cache thrash, excessive context switching.

5 TRADE-OFFS

<i>Fairness</i>	↔	<i>Latency (responsiveness)</i>
<i>Throughput</i>	↔	<i>Latency</i>
<i>Short quantum</i>	↔	<i>Switching overhead</i>
<i>High priority</i>	↔	<i>Starvation risk</i>
<i>Affinity / locality</i>	↔	<i>Load balance</i>
<i>Low variance (determinism)</i>	↔	<i>High utilization</i>
<i>Strong isolation</i>	↔	<i>Resource utilization</i>
<i>Aggressive preemption</i>	↔	<i>Cache locality</i>
<i>Complex policy</i>	↔	<i>Simplicity & predictability</i>

Rule of thumb

- Interactive workloads** → lower latency, fairness
- Batch workloads** → higher throughput, utilization
- Real-time workloads** → bounded latency, deadlines

6 DISCUSSION



Preemptive vs. cooperative: preemptive improves responsiveness; cooperative reduces overhead.



Quantum choice: too small → overhead; too large → poor responsiveness.



Priority inversion: low-prio holds lock => high-prio blocked; solve with priority inheritance/ceiling.



Starvation & aging: boost waiting tasks to ensure progress.



Interactive vs. batch: favor latency vs. throughput.



CPU-bound vs. I/O-bound: mix affects optimal policy.



Oversubscription: runnable > CPUs → queueing delay; control via limits & backpressure.



Work stealing: reduce imbalance on many-core.



Real-time guarantees: hard/soft deadlines, determinism, jitter bounds.

7 ALGORITHMS AT A GLANCE

Algorithm	Preemptive?	Key Strength	Key Weakness	Typical Domain / Use
FCFS (FIFO)	No	Simple; low overhead	Convoy effect; poor response	Batch systems
SJF (non-preemptive)	No	Minimizes avg waiting time (known bursts)	Needs burst estimate; starvation of long jobs	Batch with known/estimated bursts
SRTF (preemptive SJF)	Yes	Minimizes avg waiting time	High overhead; starvation risk	Batch/OS (theoretical optimal)
Round Robin (RR)	Yes	Fair; good response time	Higher context switch cost	Time-sharing OS, interactive systems
Priority Scheduling	Yes/No	Supports importance; flexible	Starvation of low-priority tasks	Real-time, systems with classes
Multilevel Feedback Queue (MLFQ)	Yes	Balances throughput & response	Complex tuning; starvation if mis-set	General-purpose OS
Fair-share / CFS-style	Yes	Proportional fairness over time	More complex; not strict real-time	Linux CFS, general-purpose
EDF (Earliest Deadline First)	Yes	Optimal for single CPU (EDF)	Overhead; not ideal multiprocessor	Real-time systems
Rate-Monotonic (RM)	Yes	Simple; good for periodic tasks	Less utilization than EDF	Real-time periodic workloads

8 RELATED

- Processes & threads
- Context switching
- Interrupts & traps
- Synchronization (locks, semaphores)
- Memory management (pages, TLB)
- NUMA & memory locality
- Thread pools & executors
- cgroups / control groups

9 PRODUCTION

- Linux:** CFS fair scheduling, priority classes (SCHED_FIFO/RR)
- Containers:** CPU shares, quotas, `cpu.max`, `cpuset`
- JVM:** ForkJoinPool, GC threads, biased scheduling
- Go:** M:N scheduler, GOMAXPROCS
- Databases:** query schedulers, `ioprio`, resource groups
- UI & audio:** minimize latency, avoid dropouts
- Cloud:** noisy neighbors, quotas, burst credits

10 INTERVIEW THINKING

- Identify workload & objective (latency / throughput / deadlines).
- Choose metrics (avg, p99, utilization, misses).
- Pick candidate policy/algorithms.
- Simulate with examples (Gantt).
- Check edge cases: bursts, I/O, starvation, inversion.
- Consider multicore, contention, NUMA, migrations.
- Measure & tune; iterate.

11 SENIOR NOTES

- ✓ Optimize the objective, not the average.
- ✓ Observe run-queue time (not just CPU time).
- ✓ Control oversubscription and queueing delay.
- ✓ Preserve cache & memory affinity thoughtfully.
- ✓ Use cgroups/quotas for isolation & SLOs.
- ✓ Distinguish OS scheduler vs. runtime scheduler.
- ✓ Analyze tail latency (p99/p999).
- ✓ Watch for priority inversion & lock contention.
- ✓ Measure before tuning; validate after changes.



MENTAL HOOK: "Scheduling is policy under scarcity: every choice optimizes one workload and disappoints another."

CONTEXT SWITCHING

Saving one execution context and restoring another

1. WHY



CPU multiplexing
let many tasks share few cores



Blocking
I/O or waits would stall the CPU



Preemption
enforce fairness & time slicing

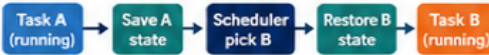


Responsiveness
meet latency targets for interactive workloads



Isolation
separate address spaces & protection domains

2. CORE IDEA



- 1 Pause current task A (preempt or block).
- 2 Save A's architectural state.
- 3 Choose next task B to run.
- 4 Restore B's state.
- 5 Resume execution of B.

TYPES OF CONTEXT SWITCH

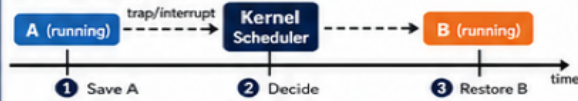
	Process Switch Different address space (different PID).
	Thread Switch Same process, different TID.
	Privilege-Mode Switch User ↔ Kernel mode (not necessarily a task switch).
	Coroutine / Fiber Switch User-space cooperative switch (often no kernel involvement).

3. INTERNALS

TRIGGERS

- Timer interrupt (quantum expiration)
- Blocking system call (e.g., read, futex wait)
- I/O completion / event wakeup
- Higher-priority task becomes runnable

HIGH-LEVEL FLOW



WHAT GETS SAVED/RESTORED (CHECKLIST)

- | | |
|---|---|
| ✓ Program Counter (PC) | ✓ Address Space (page table) |
| ✓ Stack Pointer (SP) | ✓ ASID / PCID tag |
| ✓ General Registers | ✓ TLB entries (may flush or retain via ASID/PCID) |
| ✓ Status / Flags | ✓ FPU / SIMD state (lazy or eager) |
| ✓ Kernel Stack Pointer | ✓ Other Arch State (debug regs, perf counters) |
| ✓ Scheduling / Accounting State (nice, runtime, etc.) | |

MICROARCHITECTURAL EFFECTS



4. COMPLEXITY

DIRECT COST (example ranges)

Switch Type	Cycles (approx.)
Thread Switch (same process)	~ 1K - 5K
Process Switch (same CPU)	~ 3K - 20K+
Process Switch (diff. CPU/NUMA)	~ 10K - 50K+
Mode Switch (user ↔ kernel)	~ 100 - 1K

TOTAL COST = DIRECT + INDIRECT

- Indirect: cache/TLB miss, pipeline refill, branch predictor loss, cold data/code.
- Frequency × Cost = Overhead.

⌚ Runnable queue delay adds latency (waiting for CPU).

📊 Switch rate = switches / second.

📈 Track p50 / p95 / p99 latency impact.

🔄 Voluntary switch: yield/block/I/O.
Involuntary switch: preemption/interrupt.

⚠️ A system call does not always cause a context switch (may return immediately).
A mode switch is not necessarily a task switch.

5. TRADE-OFFS

GOOD

- ✓ Responsiveness
- ✓ Fairness
- ✓ Concurrency
- ✓ Resource Sharing
- ✓ Isolation & Security



BAD

- ✗ CPU Overhead
- ✗ Locality Loss
- ✗ Cache/TLB Misses
- ✗ Jitter / Tail Latency
- ✗ Contention
- ✗ Debugging Complexity

TUNE TO BALANCE

⚙️ Better fairness / latency	↔	⚡ Higher overhead / less locality
👤 Higher concurrency	↔	🔥 More contention / jitter
🛡️ Stronger isolation	↔	💾 More state to save / memory
⌚ Frequent preemption	↔	📉 Throughput may drop
⌚ Large quantum	↔	📉 Better throughput, worse latency
⌚ Small quantum	↔	📈 Better latency, more switches

6. DISCUSSION



Process vs Thread

Process switch is costlier (address space).



Voluntary vs Involuntary

Blocking/yield vs preemption (timer/high prio).



Quantum Size

Too small → overhead; too large → poor latency.



Oversubscription

Threads > cores → more context switches.



CPU Affinity

Preserve cache warmth; avoid ping-pong.



Blocking vs Async

Blocking drives switches; async reduces them.



User-Level Scheduling

Coroutines/fibers reduce kernel switches.



Hyperthreads (SMT)

Shares core; context switches still cost caches.



NUMA Awareness

Remote memory + migration ↑ cost.



When Switches Become Excessive

High CPU in kernel, runq length > cores, p95 latency ↑, low IPC.

7. RELATED

- Scheduling Algorithms**
RR, CFS, Priority, EDF
- Processes & Address Spaces**
Fork/Exec, Copy-on-Write
- Threads & Concurrency**
POSIX threads, futex
- Interrupts**
Hardware/Software interrupts
- System Calls**
Trap to kernel
- TLB (Translation Lookaside Buffer)**
Virtual → Physical mapping cache
- CPU Cache Hierarchy**
L1/L2/L3, inclusion, coherence
- Async Programming**
Event loops, futures, epoll/io_uring

8. PRODUCTION

- | | |
|--|---|
| Web Servers
(nginx, Envoy, Apache) | Keep workers ≈ cores, reuse connections, minimize blocking, use async I/O. |
| Databases
(PostgreSQL, MySQL) | Connection pools, prepared statements, avoid context switch storms. |
| JVM / Go Runtimes | M:N scheduling, work stealing, GC safepoints cause stops. |
| Containers | cgroups, namespaces, many tenants → watch throttling & run queue. |
| Real-Time / Audio | Low latency needs: pin threads, RT priority, avoid page faults & blocking I/O. |
| Observability (Metrics) | vmstat cs, in/r; perf sched:sched_switch, perf stat context-switches; top / htop. |

KEY METRICS

- | | | | | |
|---------------------|---------------|------------------|------------------|----------------------------|
| cs context switches | in interrupts | run queue length | %sys system time | IPC instructions per cycle |
|---------------------|---------------|------------------|------------------|----------------------------|

9. INTERVIEW THINKING

- 1 Identify trigger: timer, block, wakeup, higher priority?
- 2 Determine which context changes: thread/process/mode/coroutine? Which state must be saved?
- 3 Separate direct vs indirect costs.
- 4 Inspect workload: switch rate? runq length? CPU util?
- 5 Form hypothesis: what's causing frequent switches?
- 6 Measure: vmstat, perf, trace, flame graphs, p99 latency.
- 7 Mitigate: reduce blocking, right-size threads, batch, affinity, async, improve locality.

10. SENIOR NOTES

- | | |
|--|--|
| Reduce runnable work
Do real work, not busy work. | Avoid accidental oversubscription
Match threads to cores (or slightly more). |
| Batch intelligently
Amortize overhead without hurting tail latency. | Preserve locality
Data, code, CPU affinity, warm caches. |
| Use affinity only with evidence
Don't over-constrain the scheduler. | Watch tail latency & queueing
Runq, service time, saturation. |
| Context switching is necessary but expensive—measure, then minimize pain. | |

11. MENTAL HOOK



A context switch moves no business work forward; it buys fairness and responsiveness by spending locality.

FILE SYSTEMS

Turning persistent blocks into named, reliable data

1 WHY



Persistent naming
Survive reboots and crashes



Organization
Hierarchical namespace



Sharing
Multi-user access



Protection
Permissions, ACLs, isolation



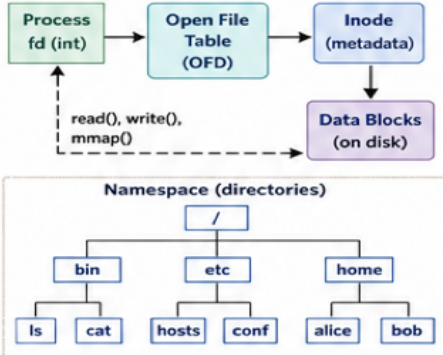
Crash recovery
Consistent after failures



Device abstraction
Hide physical device details

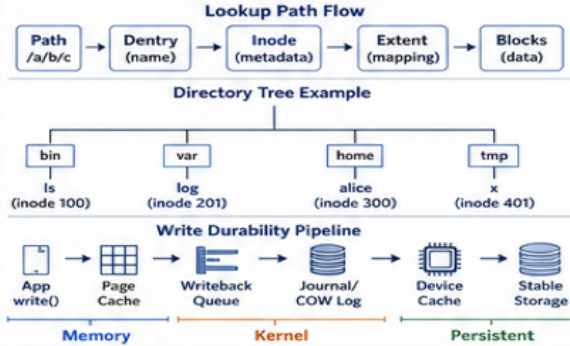
2 CORE IDEA

- Directory namespace maps names to metadata (inodes) and data blocks.
- File descriptor (fd) references an open-file description (OFD) in the open file table.
- Bytes are accessed through read/write or mmap.



3 INTERNALS

- VFS (Virtual File System) layer
- Path lookup
- Dentries (name → inode cache)
- Inodes (metadata)
- Permissions: mode, UID/GID, ACLs
- Open file table & OFDs
- Page cache (buffer cache)
- Block allocation (extents)
- Free-space metadata (bitmaps, trees)
- Extents → physical block mapping
- Buffering & readahead
- Writeback (dirty pages to disk)
- fsync: flush data + metadata
- Journaling or Copy-on-Write
- Mount points & superblock
- Hard links: multiple names → same inode
- Symbolic links: special file with path target



4 COMPLEXITY

Aspect	What Makes It Hard
Path traversal	Per-component lookup, cache misses, symlinks
Sequential vs Random I/O	Throughput vs latency, queue depth
Metadata operations	Create/unlink/rename, dir updates, link counts
Page-cache hits/misses	Cache size, locality, thrash
Read amplification	Small reads pull large blocks/pages
Write amplification	Journals, COW, metadata
Fragmentation	Extent breaks, placement, seeks
fsync tail latency	Flush, journal commit, device cache
Throughput	MB/s sustained
IOPS	Operations per second

5 TRADE-OFFS

Journaling (ordered/meta) Lower space amp, faster writes	VS	Copy-on-Write (Btrfs, ZFS, APFS) Stronger consistency, higher write amp
Consistency Strict ordering, sync more		Performance Batching, async, higher throughput
Small Blocks Less waste, more metadata		Large Blocks/Extents Less metadata, better seq I/O
Delayed Allocation Better packing, risk on crash		Immediate Allocation Faster recovery, possible fragmentation
Buffered I/O Cache helps, less syscall overhead		Direct I/O Bypass cache, aligned, app-managed
Local FS Lower latency, strong POSIX		Network FS Share across hosts, higher latency

6 DISCUSSION

1 Rename guarantees	Atomic within a directory (POSIX)
2 Crash consistency	After crash, FS is consistent (no corruption)
3 Atomicity boundaries	Single metadata ops; not multi-file tx
4 fsync semantics	Data + required metadata durable
5 Sparse files	Holes read as zero, save space
6 Permissions / ACLs	Mode bits + ACLs, inheritance
7 Inode exhaustion	No new files even if space free
8 File locking	Advisory locks, flock/fcntl/leases
9 mmap trade-offs	Page faults, coherency, TLB pressure
10 SSD implications	Erase blocks, FTL, write amp, TRIM
11 Data vs metadata durability	Both needed; order matters

7 RELATED

Block devices	Disks, NVMe, partitions, LVM
Virtual memory / Page cache	Pages, LRU, writeback
System calls	open, read, write, mmap, fsync
Databases / WAL	Durable logs, checkpoints
Object storage	S3, GCS, blobs (eventual consistency)
Distributed file systems	NFS, SMB/CIFS, Ceph, GFS
RAID	Redundancy, performance tiers
Caching layers	Page cache, CDN, proxy caches

8 PRODUCTION

Databases and WAL	fsync for WAL, data files via direct I/O
Container layered filesystems	OverlayFS, copy-up cost, inode pressure
Logs	Append-heavy, rotation, fsync strategy
Object gateways	Map files ↔ objects, consistency models
Content services	Large files, streaming, range reads
Kubernetes volumes	PVCs, CSI, RWX vs RWO, snapshots
Backups and snapshots	Incremental, dedupe, point-in-time

9 INTERVIEW THINKING

1 Clarify workload and durability needs	Ops, size, access pattern, RPO/RTO
2 Trace data & metadata path	From syscall to stable storage
3 Define failure model	Crash, power loss, media failure
4 Estimate access pattern	Seq vs rand, read/write mix, size
5 Select design / configuration	FS type, mount options, block size
6 Test crash behavior	Power-cut tests, fsync boundaries
7 Observe & iterate	Metrics, latency tails, errors

10 SENIOR NOTES

- ✓ Durable write requires explicit contract (fsync, O_DSYNC).
- ✓ Benchmark realistic cold & warm paths, not micro-bench only.
- ✓ Directories can become hotspots; scale or shard.
- ✓ Avoid assuming close() equals durable.
- ✓ Account for quotas, inodes, and reserved space.
- ✓ Design recovery and verification before optimization.

11 MENTAL HOOK



A file system is a database for bytes: names, metadata, allocation, caching, and recovery all shape correctness.

PROCESSES

The operating system's unit of isolation and execution

1 WHY



Fault containment
Crash stays in the process



Security boundary
Separate identity & access



Resource ownership
Memory, CPU, I/O, files



Concurrency
Many independent activities



Lifecycle management
Create, run, stop, reap

2 CORE IDEA

A process is a running program with a **private virtual address space**, execution threads, credentials and kernel-managed resources.

PROGRAM

Passive code on disk (ELF, DLL)



PROCESS

Active instance with resources and state



THREAD

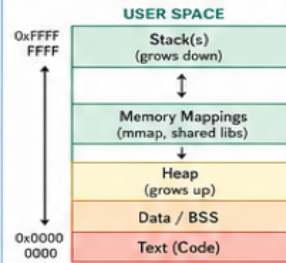
Execution path within a process (stack + TID)



One program can have many processes.
One process can have many threads.

3 INTERNALS

PROCESS ANATOMY (per process)



- PCB / Task Structure
- PID / PPID
- State (see below)
- Threads (TIDs)
- CPU Context
- Page Tables (per address space)
- File-Descriptor Table
- Signal Handlers / Mask
- Credentials (UID, GID, caps)
- Environment Variables
- Working Directory
- Resource Limits (rlimits)
- Accounting / Stats
- IPC Handles / Sockets / Pipes

STATES (lifecycle)



4 COMPLEXITY

Factor Notes / Order of Magnitude

	Creation cost	fork() ~ O(#VM pages) metadata spawn/exec ~ more (load + init)
	Copy-on-write	Cheap at fork; cost when pages are written
	Address space & page tables	Per-process page tables; more memory & TLB pressure
	Context switch	Save/restore CPU state ~ thousands of cycles
	IPC overhead	Pipes/Unix sockets < TCP < message brokers / shared mem
	Startup time	Cold start > warm start > pre-fork
	Memory footprint	Code + heap + stacks + page tables + kernel structs
	FD limits	Per-process ulimit (e.g., 1024/65535)
	Process count	Kernel pid_max; PID namespace limits
	Queueing	Run queue length ↑ = latency ↑

5 TRADE-OFFS

Benefit (Pro)	Cost (Con)
Strong isolation	Higher memory usage
Simple failure model	Slower startup
Security boundary	Expensive communication
Independent lifecycles	More kernel overhead
Language/runtime agnostic	Harder fine-grained sharing

COMMON CHOICES

Process-per-request

✓ Isolation, crash safety ✗ High churn, latency, memory

Process pools / pre-fork

✓ Amortized startup, stable ✗ Capacity planning, idle cost

Processes vs Threads vs Containers

Unit	PROCESS	THREAD	CONTAINER
Isolation	Strong	Weak	Medium (OS isolated)
Memory Sharing	No (COW)	Yes (same address space)	Yes (within container)
Startup	Slower	Fast	Medium
Failure Impact	Contained	Can crash whole proc	Contained (to container)
Use When	Safety, privilege separation	High perf, fine sharing	Multi-tenant, deploy units

6 DISCUSSION

- fork() semantics** Duplicates process; child gets own address space (COW)
- fork in multithreaded apps** Only calling thread survives in child; avoid locks
- exec()** Replaces process image; keeps PID, many attrs
- Zombies** Child exited, not reaped; consumes PID & slots
- Signals** Async notifications; default / ignore / handle
- Daemonization** Detach from TTY; setuid(0); double-fork; umask
- PID namespaces** PID 1 inside container; isolates process trees
- Permissions & credentials** UID, GID, capabilities, setuid/setgid, ambient caps
- Resource limits** ulimit / rlimits: CPU, memory, FDs, processes, etc.
- Process groups & sessions** Job control; kill by group; controlling terminal
- Graceful shutdown** SIGTERM first; drain; timeout; SIGKILL last
- Crash recovery** Restart policies; core dumps; backoff strategies

7 RELATED

Threads	Scheduling	Context Switching
System Calls	Virtual Memory	IPC
Signals	File Descriptors	Containers / cgroups

8 PRODUCTION

- Web servers / workers** Gunicorn pre-fork, Node cluster
- Databases** PostgreSQL backends, MySQL processes
- Supervisors / Service Managers** systemd, supervisord, launchd, Windows SCM
- Browsers** Multi-process: renderer, GPU, network, broker
- Build systems** Make, Ninja, Bazel: parallel worker processes
- Shells / Pipelines** Pipelines connect processes via pipes
- Containers & Job Runners** Kubernetes pods, Docker containers, CI runners

9 INTERVIEW THINKING

- 1 **Clarify the unit:** What is isolated? What is shared?
- 2 **Map resources owned:** memory, FDs, CPU, creds, limits, cwd, env...
- 3 **Trace lifecycle:** How created? How terminates? Who reaps?
- 4 **Identify sharing & communication:** IPC, shared memory, files, sockets
- 5 **Model failures:** crashes, kills, timeouts, zombies, orphans
- 6 **Measure costs:** startup, context switch, memory, IPC
- 7 **Choose concurrency unit:** process vs thread vs container based on workload and isolation needs

10 SENIOR NOTES

- ✓ Always reap children; prevent zombies and PID leaks.
- ✓ Close or set CLOEXEC on inherited file descriptors.
- ✓ Define clear shutdown contract: drain, timeout, kill.
- ✓ Bound resources: ulimits, cgroups, quotas, backpressure.
- ✓ Avoid unsafe post-fork work in multithreaded programs.
- ✓ Isolate untrusted code in separate processes/containers.
- ✓ Observe & alert: restarts, memory growth, FD leaks.
- ✓ Prefer idempotent start/stop and health checks.

11 MENTAL HOOK



A process packages execution with an ownership boundary: memory, resources, identity, and failure.

INTER-PROCESS COMMUNICATION (IPC)

Moving data and coordination across isolation boundaries

1 WHY



Process isolation prevents direct memory access.



Systems need explicit **data exchange**, **synchronization**, **event notification** and **ownership transfer**.



Improves reliability, security, fault containment and modularity.



Enables communication across crashes, machines and privilege boundaries.

2 CORE IDEA



Choose a communication channel and protocol.

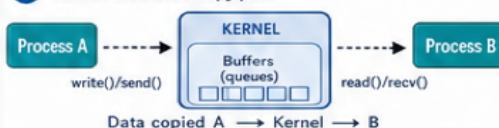
Framing	Lifecycle
Serialization	Security
Ordering	Failure semantics
Backpressure	Ownership transfer

Define the contract both sides implement.

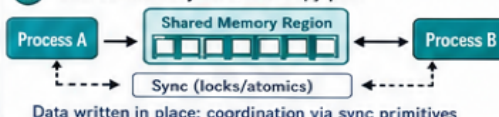
3 INTERNALS

Two fundamental data paths

A Kernel-mediated copy path



B Shared-memory zero/low-copy path



INTERNALS (CONT.) – IPC PRIMITIVES COMPARISON

Primitive	Directionality	Local / Remote	Message / Stream	Copying	Synchronization	Strengths	Risks / Limitations
Anonymous Pipes	Unidirectional	Local	Byte Stream	Yes (Kernel copy)	Implicit (kernel buffer)	Simple, fast for parents+children	Unidirectional, no random access
Named Pipes (FIFOs)	Unidirectional (or bidir with 2)	Local	Byte Stream	Yes	Implicit (kernel buffer)	Persist in FS, works across unrelated procs	FS namespace cleanup needed
Unix Domain Sockets	Bidirectional	Local	Stream or Datagram	Yes	Kernel buffers, ancillary data	Full-duplex, fd passing, creds	Slightly higher overhead than pipes
TCP Sockets	Bidirectional	Local / Remote	Byte Stream	Yes	TCP (flow control, retransmission)	Works across networks, reliable, mature	Higher latency, congestion, Nagle/MTU issues
Message Queues (MQ)	Typically Bidirectional	Local / Remote*	Message	Yes	Kernel queue, priorities	Discrete messages, delivery semantics	Queue limits, latency variability
Shared Memory + Synchronization	Bidirectional	Local	Memory Region + Signals	No (zero-copy)	Locks, semaphores, atomics, condvars	Highest throughput, low latency	Complexity, race conditions, cache coherence
Signals / Events	Unidirectional (Async)	Local	Signal (no payload)	No	Kernel signal queue	Low overhead notifications	Limited payload, coalescing, race prone
Memory-Mapped Files (mmap)	Bidirectional	Local or Shared FS	Memory Region	No (page cache)	File locks / atomics (optional)	Large data, random access, persistence	Disk latency, I/O variability
RPC (Higher-Level Abstraction)	Typically Bidirectional	Local / Remote	Message (Request/Reply)	Usually Yes (or Zero-copy with care)	Protocol-level (timeouts, retries)	Developer productivity, contract, tooling	Adds a layer, latency overhead

*Some message queues support remote via brokers (e.g., RabbitMQ, NATS).

4 COMPLEXITY



Cost components: syscalls, copies, serialization, context switches, queueing, cache misses.



Measure: throughput (msg/s, MB/s) and latency (p50 / p99 / p999).



Buffering: size affects throughput and tail latency.



Shared memory: fastest data path; requires locks/atomics and cache coherence.



Unix domain sockets usually lower latency than TCP on same host.



Batching and amortization reduce per-message costs (e.g., `writew`, `sendmmsg`, batching).

5 TRADE-OFFS

Simplicity	↔	Speed / Performance
Isolation	↔	Sharing / Throughput
Copy Cost	↔	Synchronization Complexity
Reliable Delivery	↔	Lower Latency
Blocking (Easy)	↔	Async (Complex)
Coupling (Tight)	↔	Evolvability (Loose)
Ordering Guarantee	↔	Concurrency / Parallelism
Strong Consistency	↔	Availability / Partition Tolerance
Small Messages	↔	Large Payload Efficiency

6 DISCUSSION

- ✓ Message boundaries vs byte streams (framing needed).
- ✓ Partial reads/writes: always possible—loop until complete.
- ✓ Backpressure: flow control, credit-based, or drop/queue.
- ✓ Dead peers: detect (EOF, RST, heartbeats) and recover.
- ✓ Timeouts & cancellation: fail fast, retry with backoff.
- ✓ Ordering: per-connection vs global; FIFO not guaranteed.
- ✓ Delivery semantics: at-most-once / at-least-once / exactly-once.
- ✓ Descriptor passing (Unix sockets): pass fds, sockets.
- ✓ Permissions & identities: UID/GID, ACLs, capabilities.
- ✓ Race conditions: TOCTOU, shared state, locking discipline.
- ✓ Cleanup: `close()`, `unlink()`, `shm_unlink()`, handle crashes.
- ✓ Versioning: message/schema evolution, compatibility.
- ✓ Large payloads: chunking, compression, zero-copy (`sendfile`, `splice`, `mmap`, shared memory).

RELATED

- Processes:** `fork()`, `exec()`, `exit()`, `wait()`, signals
- System Calls:** `read()`, `write()`, `send()`, `recv()`, `poll()`, `epoll()`
- Sockets:** `AF_UNIX`, `AF_INET`, TCP, UDP
- Shared Memory:** `shm_open()`, `mmap()`, `memfd_create()`
- Synchronization:** `mutexes`, `semaphores`, `atomics`, `futex`
- Serialization:** JSON, `Protobuf`, `FlatBuffers`, `Cap'n Proto`
- Event Loops:** `select()`, `poll()`, `epoll()`, `kqueue()`, `IOCP`
- RPC:** **RPC Frameworks:** `gRPC`, `Thrift`, `Cap'n Proto RPC`, `Dubbo`
- Message Queues:** `POSIX MQ`, `RabbitMQ`, `Kafka`, `NATS`

8 PRODUCTION

- Shell pipelines, ETL and CLI tooling
- Browser multiprocess (renderer ↔ browser ↔ GPU)
- Databases (client ↔ server, WAL, replication)
- Service mesh sidecars and proxies
- Local agents and daemons (metrics, logging, auth)
- Container runtimes (containerd, Docker)
- Media pipelines (capture → encode → stream)
- High-frequency trading (low-latency IPC)

9 INTERVIEW THINKING

- 1 Clarify topology: local vs remote, processes, languages.
- 2 Define payload: size, rate, direction, patterns (burst/steady).
- 3 Define semantics: ordering, reliability, idempotency, delivery.
- 4 Quantify: latency targets (p50/p99), throughput, concurrency.
- 5 Pick primitive: match constraints to the right tool.
- 6 Design protocol: framing, serialization, backpressure, security, versioning.



Test failure modes: crash, partitions, resource limits.
Observe: metrics, tracing, logs; iterate and harden.

SENIOR NOTES

- ☐ Protocol is the real API—design it deliberately.
- ☐ Bound every queue and buffer; plan for overload.
- ☐ Handle partial I/O everywhere; never assume all-or-nothing.
- ☐ Version messages and schemas; support rollouts.
- ☐ Authenticate and authorize peers; enforce least privilege.
- ☐ Define ownership and lifecycle; make cleanup idempotent.
- ☐ Prefer explicit over implicit; document assumptions.
- ☐ Measure end-to-end tail latency, not just averages.
- ☐ Simplicity first—optimize only with real bottlenecks.

11 MENTAL HOOK

IPC trades isolation for coordination—the channel is easy; the protocol and failure semantics are the design.